Lab 3.1: Basic Matrix Multiplication

1. Objective

The purpose of this lab is to implement a basic dense matrix multiplication kernel

2. Preliminary work

Step 1: Log in to the GPU side of Roihu (see Lab 1 if the login is refused because your SSH certificate has expired):

```
ssh <username>@roihu-gpu.csc.fi
```

Step 2: If you have not already cloned the lab repository in Lab 1, do it now, in the directory where you would like to put it:

```
git clone https://github.com/jolilius/GP26_Labs.git
cd GP26_Labs
```

You are now ready to begin the lab.

3. Procedure

Step 1: Update your local repository to obtain the code needed for the assignment.

```
cd GP26_Labs
git pull
cd Lab_3.1
```

Step 2: Edit the file main.cu to implement the following where indicated:

1. Allocate device memory
2. Copy host memory to device
3. Copy results from device to host
4. Free device memory

Step 3: Edit the file kernel.cu to initialize the thread block and kernel grid dimensions and invoke the CUDA kernel, and to implement the matrix multiplication kernel code.

Step 4: Compile and test your code. Programs that use a GPU run on a compute node in a Slurm job, so you submit `run.sh`, which builds the program on the node and runs it. Use the CSC project you joined in Lab 1 as the account:

```
sbatch --account=<your-project> run.sh             # Uses the default matrix sizes
sbatch --account=<your-project> run.sh <m>         # Uses square m x m matrices
sbatch --account=<your-project> run.sh <m> <k> <n> # Uses (m x k) and (k x n) input matrices
```

The output is written to `lab3.1_<jobnumber>.out` and to `output.txt` (each job overwrites `output.txt`). Wait for the job to finish (`squeue --me` shows your jobs) and then read the file. A correct solution prints something like the following (the timings will differ):

```
Setting up the problem...0.054154 s
    A: 1000 x 1000
    B: 1000 x 1000
    C: 1000 x 1000
Allocating device variables...0.797001 s
Copying data from host to device...0.000064 s
Launching kernel...0.000472 s
Copying data from device to host...0.000345 s
Verifying results...TEST PASSED
```

Compile errors show up in `lab3.1_<jobnumber>.out`.

It is a good idea to test and debug initially with small input dimensions. Your code is expected to work for varying input dimensions – which may or may not be divisible by your block size – so don't forget to pay attention to boundary conditions. Run the default sizes last, so that `output.txt` holds the output for the default sizes.

Step 5: Answer the following questions in a new file named answers.txt:

1. How many times is each element of each input matrix loaded during the execution of the kernel?
2. What is the memory-access to floating-point computation ratio in each thread? Consider multiplication and addition as separate operations, and ignore the global memory store at the end. Only count global memory loads towards your off-chip bandwidth.

Step 6: Submit your assignment. You need to submit the following files:

- main.cu
- kernel.cu
- answers.txt
- output.txt (the output of your run, which must show `TEST PASSED`)

Compress the files and name them after your student name and student id like so:

```
Lab3.1_Name_id.tar
```

Upload the file to the assignment in Moodle.

4. Grading

Your submission will be graded based on the following criteria.

- Functionality/knowledge: 65%
  - Correct code and output results (`output.txt` shows `TEST PASSED`)
  - Correct usage of CUDA library calls and C extensions
  - Correct handling of boundary cases
- Answers to questions: 35%
  - Correct answer to questions in step 5
  - Sufficient work is shown
  - Neatness and clarity